

```
>>> def square(x):
...     return x*x
>>> alist=[1,2,3,4,5]
>>> map(square, alist)
[1, 4, 9, 16, 25]
```

The same output can be achieved with list comprehension:

```
>>> [square(element) for element in alist]
[1, 4, 9, 16, 25]
```

Filter

As the name suggests, *filter* returns those elements that satisfy a particular condition. For example, the following code filters out all the even numbers:

```
>>> def iseven(x):
...     return x%2 == 0
>>> alist=[1,2,3,4,5]
>>> filter(iseven, alist)
[2, 4]
```

The same output can be achieved with list comprehension as follows:

```
>>> [x for x in alist if iseven(x)]
[2, 4]
```



Note: *map()* and *filter()* are somewhat obsolete, but they have been explained since they are the fundamental concepts of functional programming. The *itertools* module provides the functions *imap()* and *ifilter()*, which perform the same function. Readers are encouraged to refer to the Python documentation for more on *itertools*.

Lambda

The *lambda* statement allows one to create functions on the fly, i.e., functions that are not bound to a name. It is most commonly used with *map()*, *filter()*, etc. Lambda can be used to replace several small functions, and thus save a lot of unnecessary code. Consider the *iseven()* function defined above. It can be replaced by a lambda statement as follows:

```
>>> alist=[1,2,3,4,5]
>>> filter(lambda x: x % 2 == 0, alist)
[2, 4]
```

Similarly, the code example for *map()* can be rewritten as:

```
>>> alist=[1,2,3,4,5]
>>> map(lambda x: x * x, alist)
[1, 4, 9, 16, 25]
```

As an example of how useful and powerful the *lambda* statement can be, here is the implementation of 'Sieve of Eratosthenes':

```
>>> numbers=range(2,50)
>>> for i in range(2,8):
...     numbers=filter(lambda x: x == i or x % i, numbers)
>>> numbers
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
```

Thus, the *lambda* statement can be combined with *map()*, *filter()*, etc, to produce numerous powerful computations in very small statements. *Lambda* is rather limited in the functions that it can define; put too much into a *lambda* statement, and it'll get complicated and hard to understand what is going on. Notice that without *lambda*, the code below is much easier to understand:

```
total = reduce(lambda a, b: (0, a[1] + b[1]), items)[1]

total = 0
for a, b in items:
    total += b
```

Thus, it's best to use *lambda* sparingly, and with discretion.

Future of functional programming

Although this approach has several problems related to CPU efficiency, memory efficiency, etc, it definitely has a good future, due to the fact that it plugs many loopholes in currently existing paradigms. Also, since it avoids program states, it is believed to be best suited for concurrent programming, which is very important these days due to the advent of multicore processors. Ultimately, it's a trade-off between efficiency and guaranteed performance—functional programming, although inefficient, ensures proper performance, while the other approaches have a high degree of unpredictability in the states they will assume. We must wait and see which comes out on top. 

References

- [1] http://en.wikipedia.org/wiki/Functional_programming
- [2] <http://docs.python.org/howto/functional.html>
- [3] http://www.secnexit.de/oll/Python/lambda_functions.hawk

By: Arvind S Raj

The author is an avid open source enthusiast and advocate and is interested in computer security. He loves learning new technologies and practical applications of theoretical ideas.